HERMES · PHASE 1 · LESSON 0008 · LAB EXERCISE 07-TOOL-LOOP · ~45–60 MIN

Tool Use, the Loop's Heartbeat

Exercise 06 sent one request and read one answer. What should the supervisor do when the answer is a question?

Why this matters for Hermes

Scenario step S5 (the loop iterates) is the job supervisor's core. The model asks for a tool, Hermes decides whether to run it, and the result goes back. Exercise 06 could never reach that step, because it made exactly one call. Today the boundary rule covers a third input: the arguments a model chose. The spec's `allowedTools` field gets its first reader.

1 · The reply that asks a question

Open exercise 06's `supervisor.ts`. It calls `gateway.complete` once, reads the outcome, and returns a report. One job is one request and one answer.

An audit cannot work that way. The model has no access to the atlas graph. It can write about a health report it never saw, or it can ask for one. The API lets it ask, and Hermes answers in the next request.

The model still runs nothing. A reply can request a tool by name. Hermes owns the code, the decision, and the result.

Layer	Changed?	Evidence
the endpoint ①	no	one `POST /v1/messages` per model call
the six responsibilities ①–⑥	no	the SDK still runs all six, below the port
the request body ④	yes	a `tools` array, and a `messages` array that grows
the reply's content ④	yes	a second kind of block, and a fourth `stop_reason`
the port	yes	`ModelCall.transcript` replaces exercise 06's `prompt`
who runs a tool	no	Hermes does. The provider never did and still does not

New today.

Four ideas are new.

- A reply can ask for a tool instead of finishing.
- The answer travels back as a block keyed to the question.
- The API remembers nothing, so Hermes resends the whole conversation.
- A short tool list is an offer, and an offer is not enforcement.

Reprise: narrowing a union on its tag, the boundary parse of lesson 0005, and `allowedTools` from lesson 0007.

2 · Three blocks on the wire

One word does six jobs from here on, so settle the six before reading any wire.

Where you read "tool"	What it means there
`tools`, in the request	the list Hermes declares this job may use
a `tool_use` block	one request from the model, naming a tool and its arguments
a `tool_result` block	Hermes's answer to one such request
`allowedTools`, in the spec	the names the operator permitted for this job
`tools.ts`	the catalogue: descriptions, schemas, and the code that runs
`from: "tools"`	the transcript turn that carries results back

Hermes declares its tools in the request. Each entry carries a name, a description, and an `input_schema`, which is JSON Schema. The description and the schema are all the model has to work from, so both are part of the interface.

```
"tools": [{
  "name": "graph_health",
  "description": "Report health counters for one graph: nodes, orphans and stale edges.",
  "input_schema": {"type": "object",
                   "properties": {"graph": {"type": "string", "minLength": 1,
                                            "description": "The graph to inspect…"}},
                   "required": ["graph"]}
}]
```

The lab writes that schema once, in Zod, and derives three readers from it. The call to `z.toJSONSchema()` returns a plain object at startup, and `offerTools` keeps the three fields above. The same Zod schema then parses the arguments that come back, and types the tool's own body. That is lesson 0005's single source of truth, aimed at a third boundary.

A *tool_use* block is the model's request to run one named tool, with arguments. It arrives inside an ordinary assistant message, next to any text.

```
"content": [
  {"type": "text", "text": "I need the graph health report first."},
  {"type": "tool_use", "id": "toolu_mock_0001",
   "name": "graph_health", "input": {"graph": "atlas"}}
],
"stop_reason": "tool_use"
```

A *tool_result* block carries what the tool returned, keyed to the request it answers. The `tool_use_id` repeats the id above. Without that key the provider cannot pair the two.

```
{"role": "user", "content": [
  {"type": "tool_result", "tool_use_id": "toolu_mock_0001",
   "content": "graph=atlas nodes=1284 orphans=3 stale_edges=2", "is_error": false}
]}
```

Read the role on that message. The wire has no tool role, so a result rides inside a user turn, and the conversation still alternates. The domain calls the turn `from: "tools"`, and `anthropic-gateway.ts` is the only file that knows the wire's spelling.

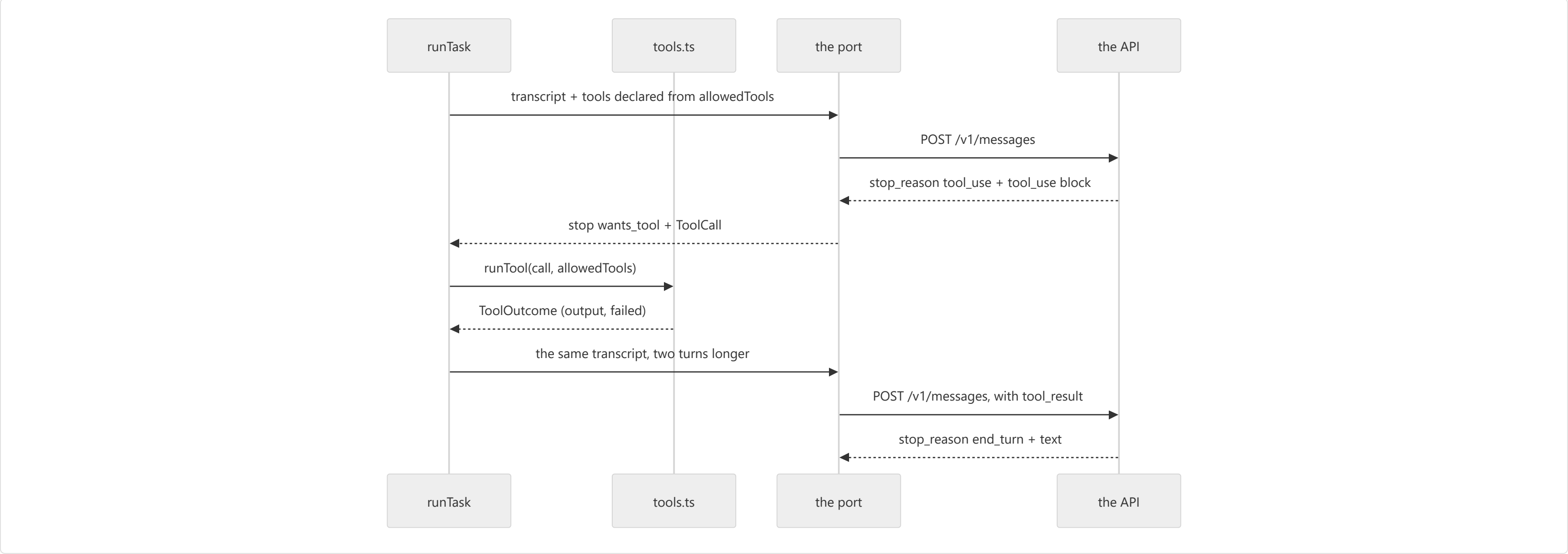
runTask
tools.ts
the port
the API
transcript + tools declared from allowedTools
POST /v1/messages
stop_reason tool_use + tool_use block
stop wants_tool + ToolCall
runTool(call, allowedTools)
ToolOutcome (output, failed)
the same transcript, two turns longer
POST /v1/messages, with tool_result
stop_reason end_turn + text

Diagram 1. One tool call, answered. Part A makes two model calls for one job.

3 · The loop, and what it costs

The *tool loop* repeats until a reply stops asking. In the lab it is one call inside a `for`, and two turns join the transcript each time round. A turn is a value of this type, declared in `gateway.ts`:

```
export type Turn =
  | { from: "operator"; text: string }
  | { from: "model"; text: string; calls: ToolCall[] }
  | { from: "tools"; results: ToolOutcome[] };
```

Part A's transcript, at the moment of the second request:

```
[ { from: "operator", text: "Summarize the atlas graph health check…" },
  { from: "model",    text: "I need the graph health report first.",
                      calls: [{ id: "toolu_mock_0001", name: "graph_health",
                                input: { graph: "atlas" } }] },
  { from: "tools",    results: [{ id: "toolu_mock_0001", failed: false,
                                  output: "graph=atlas nodes=1284 orphans=3 stale_edges=2" }] } ]
```

Three arms, narrowed on `from`, which is the union shape you met in lesson 0005 and again in lesson 0006. The adapter turns each arm into a message, and only the adapter knows which role.

```
const result = await gateway.complete({ transcript, maxTokens: spec.maxTokens, tools }, options);
// …failure handling unchanged from exercise 06…
transcript.push({ from: "model", text: reply.text, calls: reply.calls });
if (reply.stop !== "wants_tool") return report("landed", modelCall, [ … ]);

const results = reply.calls.map((call) => runTool(call, spec.allowedTools));
transcript.push({ from: "tools", results });
```

The model's turn is pushed before the tools run. Drop it and the next request answers a question the provider has no record of. One turn carries every result, in the order the model asked, because a reply may hold several `tool_use` blocks.

The transcript belongs to Hermes. The API keeps no memory between requests, so every model call resends the whole conversation, and the tools array with it. Part A measures what the transcript costs. Request 1 reports `input_tokens: 23` and request 2 reports 110, for the same job. The mock estimates from the messages alone, so that rise is the transcript. A real provider bills the tool declarations as well.

`allowedTools` now has two readers. It decides which tools are declared, so an unlisted tool never reaches the model. It is also checked again when a call comes back, because a reply is model output and can name anything at all. Part B measures that second check.

```
graph_writeback → refused        // in the catalogue, absent from allowedTools
turns sent per model call: [ 1, 3 ]
  model call 1: operator
  model call 2: operator → model → tools
```

The refusal travels as a `tool_result` with `is_error` set, and the model tries something else. A refused tool is an answer, not a crash.

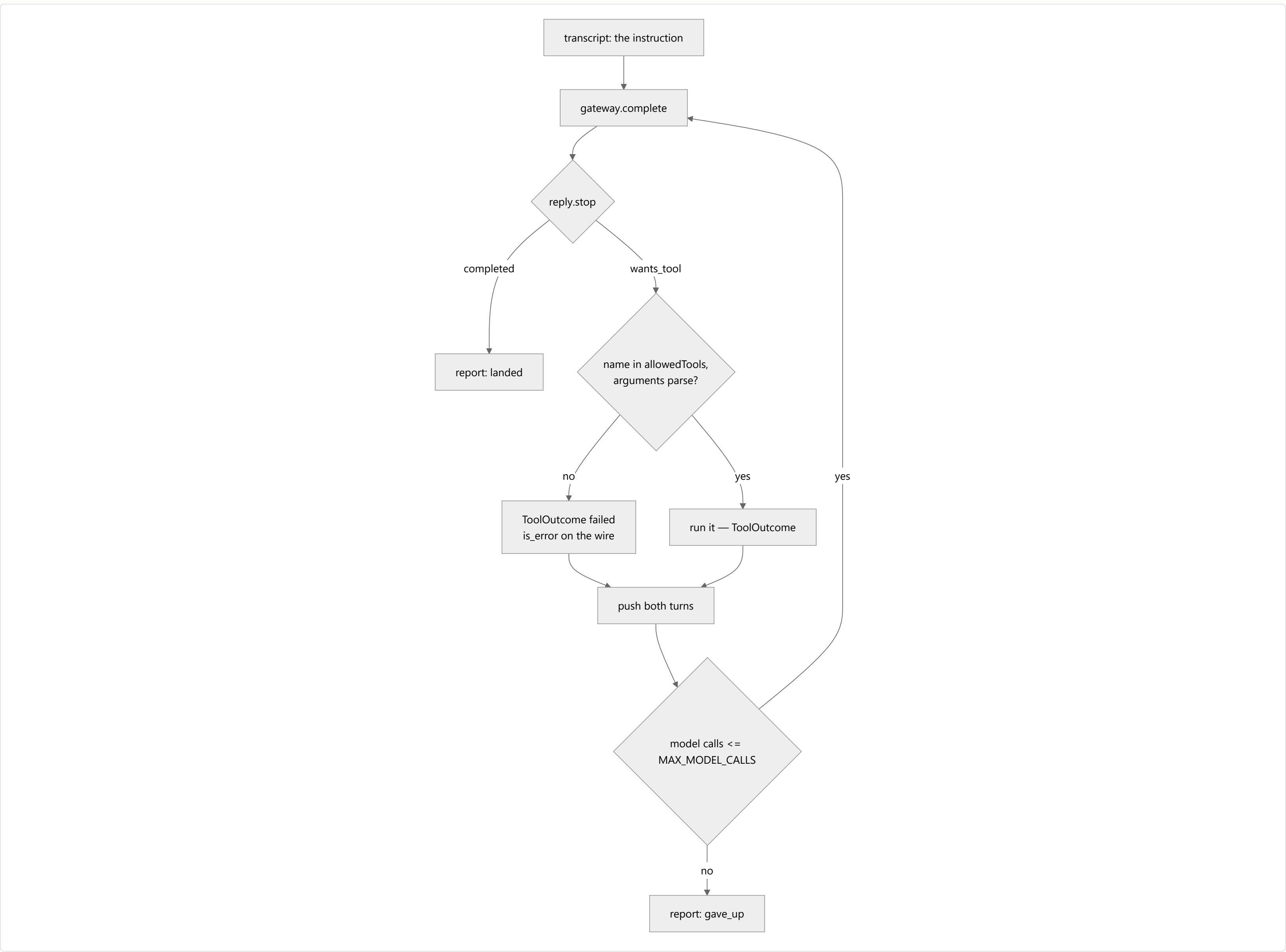
transcript: the instruction
gateway.complete
reply.stop
completed
wants_tool
report: landed
name in allowedTools, arguments parse?
no
yes
yes
ToolOutcome failed is_error on the wire
run it — ToolOutcome
push both turns
model calls <= MAX_MODEL_CALLS
no
report: gave_up

Diagram 2. Where the loop can end. Lesson 0009 replaces the literal cap with the spec's budget.

Capability	Status	Where it lives, or when it arrives
A model call can request a tool	implemented	`gateway.ts`, and the adapter's boundary parse
Hermes runs the tool and answers	implemented	`tools.ts`, measured in Parts A and B
A tool list per job	implemented	`offerTools(spec.allowedTools)`
Real evidence behind a tool	prepared	`graph_health` answers from canned data. Phase 2 serves the graph over MCP
A bound on the loop	prepared	`MAX_MODEL_CALLS` is a literal. Lesson 0009 makes it a budget and a deadline
An operator approves a tool call	planned	No code queues anything for a human. That is lesson 0010

4 · The exercise

Commands and the measured table live in `07-tool-loop/README.md`. The arc:

1. Run Part A with the mock's terminal visible. Read its two log lines.
2. Add `"graph_writeback"` to the spec's `allowedTools`, then run Part A again.
3. Set `allowedTools` to `["graph_delete"]`, a name nothing implements. Predict the two lines Part A prints before you run it.
4. Run Part C, and say what its 65 tokens have in common with exercise 06.
5. Add a third tool to the catalogue: one Zod schema, one description, one body.
6. Comment out the line that pushes the model's turn. Read the 400, and say which check caught it.

5 · Exercise: who owns each step?

Place each step with the layer that performs it

Answer from memory before you click. Feedback is immediate.

Choosing which tool to ask for, and with what arguments.

the model | Hermes, above the port | the adapter, below the port | nobody yet

Running the tool and producing its output.

the model | Hermes, above the port | the adapter, below the port | nobody yet

Deciding that a tool result travels in a `user` message.

the model | Hermes, above the port | the adapter, below the port | nobody yet

Deciding which tools the model is told about.

the model | Hermes, above the port | the adapter, below the port | nobody yet

Holding a tool call until an operator approves it.

the model | Hermes, above the port | the adapter, below the port | nobody yet

Translating `stop_reason` into `wants_tool`.

the model | Hermes, above the port | the adapter, below the port | nobody yet

Score: 0 / 0

6 · Quiz

Four questions

Q1. Part A measured `input_tokens` at 23, then 110. Why did it rise?

- The tool's output is billed at a higher rate than ordinary text
- The second request resent the whole transcript, the tool result included
- The provider adds a surcharge to any request that declares a tool
- The second call raised maxTokens, so the ceiling moved with it

Q2. A reply asks for a tool the spec never permitted. What does the lab do?

- The adapter drops the block, because the schema rejects the name
- The supervisor throws, because a bad name is an unexpected outcome
- Nothing catches it, because the tool list was already filtered
- It refuses the call and reports the refusal as a failed tool result

Q3. Which message role carries a `tool_result` block, and what pairs it?

- The user role, paired by the tool_use_id it repeats
- The tool role, paired by the position of the block
- The assistant role, paired by the name of the tool
- The user role, paired by the request id of the call

Q4. What does the provider do when a reply contains a `tool_use` block?

- It runs the named tool and includes the output in the same reply
- It waits for the tool to answer, holding the request open
- Nothing. It has returned a reply, and the next move is Hermes's
- It retries the call until the client declares a matching tool

Score: 0 / 0

7 · Say it in your own words

Answer these in the chat after the lab, with no notes.

1. Explain why the transcript lives in Hermes rather than at the provider.
2. A reviewer says the name check in `runTool` is redundant, because unlisted tools are never declared. Answer in two sentences.
3. Name the three boundaries a tool call crosses, and say what parses each one.

If you keep only four sentences from this lesson, keep these. A reply can ask for a tool, `stop_reason` reads `tool_use`, and Hermes runs the tool rather than the provider. The answer goes back as a `tool_result` block, keyed by `tool_use_id`, inside a user message. The API remembers nothing, so every model call resends the transcript and pays for it again. The spec decides which tools are declared, and Hermes checks the name again when the call comes back.

Primary source

Read the Anthropic tool use overview, then its page on handling stop reasons. Compare the loop they describe with the one in `supervisor.ts`, and note which bounds they leave to you.

Version pin. Every excerpt and number here was verified against `@anthropic-ai/sdk` 0.113.0 and `zod` 4.4.3, by live runs against the exercise 01 mock. The wire capture came from a logging proxy in front of it. The mock cannot reason, so it always asks for the first declared tool and invents arguments from that tool's schema. The block shapes are faithful to the API. The choice of tool is a stand-in. The three new terms have notes under `wiki/terms/`.

Next. Lesson 0009, *Bounds and Termination*, in the same lab. The iteration cap becomes a token budget and a deadline, and lesson 0004's mid-stream abort becomes the thing that enforces them.